Guide to HLS-Based Hardware Acceleration for Post-Quantum Cryptography Polynomial Operations

Henrique Gregory Gimenez[1]

Universidade de São Paulo, São Paulo, Brazil
henriquegregory@usp.br

Abstract. The rapid advancement of quantum computing poses a significant threat to classical public-key cryptography, prompting the standardization of Post-Quantum Cryptography (PQC) algorithms such as CRYSTALS-Kyber. Because Kyber relies heavily on polynomial ring operations that incur high computational costs, accelerating these primitives using dedicated hardware is crucial for efficiency. This guide presents a step-by-step methodology for utilizing High-Level Synthesis (HLS) tools to design and deploy a hardware accelerator for Kyber-768 polynomial operations on Field-Programmable Gate Arrays (FPGAs). Targeted at the PYNQ-Z2 SoC, the work details the translation of C reference code into hardware using AMD Vitis HLS and Vivado. We apply a Load-Compute-Store architecture alongside critical optimizations, including array partitioning, loop pipelining, and unrolling. Benchmarking ten distinct operations against an ARM Cortex-A9 software implementation, the HLS-based accelerator achieves a performance speedup of up to 9.87x for complex vector operations in the NTT domain. Furthermore, this document provides a comprehensive analysis of hardware resource utilization, power consumption, and thermal stability, serving as a practical resource for rapid prototyping and validation of PQC architectures.

Keywords: Post-Quantum Cryptography · CRYSTALS-Kyber · High-Level Synthesis (HLS) · FPGA · Hardware Acceleration · Polynomial Operations · Number-Theoretic Transform (NTT)

To follow this guide, the source code is available on GitHub at: https://github.com/henriquegreg/hls-pqc-polynomial-guide/tree/main

1 Introduction

1.1 Context and Motivation

The rapid advances in quantum computing represent a significant threat to modern public-key cryptographic schemes based on classical mathematical problems, such as integer factorization and discrete logarithms. Quantum algorithms introduced by Shor [7] and Grover [3] for factorization, discrete logarithms, and search could render these schemes vulnerable if realized in practice.

Thus, the National Institute of Standards and Technology (NIST) conducted a standardization process for Post-Quantum Cryptography (PQC) algorithms [5], resulting in the selection of CRYSTALS-Kyber for key encapsulation. Kyber is based on the Module Learning With Errors (MLWE) problem and relies heavily on operations with polynomials and polynomials vectors, which can lead to high computational costs [6].

Accelerating these mathematical primitives using dedicated hardware, such as Field-Programmable Gate Arrays (FPGAs), can achieve on high efficiency and low power consumption in embedded systems. Traditionally, hardware design at the register-transfer level (RTL) involves a slow development cycle. Since PQC standards are still evolving, High-Level Synthesis (HLS) tools, such as Vitis HLS and Vivado, offer higher abstraction and increased productivity [4]. These tools allow algorithms written in C/C++ to be translated into hardware description languages like Verilog or VHDL, facilitating FPGA implementation [1].

Therefore, documenting HLS implementations for FPGAs in post-quantum algorithms is crucial for advancing research in parallel computing and enabling rapid prototyping and validation of PQC architectures.

1.2 Goal

This guide aims to present a step-by-step methodology for using High-Level Synthesis (HLS) tools to develop hardware modules that accelerate polynomial and polynomial vector operations in the reference implementation of the Kyber-768 post-quantum cryptographic scheme. The Kyber reference implementation can be downloaded from the following link: https://github.com/pq-crystals/kyber/tree/main.

The process begins with the C implementation and progresses to physical deployment on the PYNQ-Z2 board. The PYNQ-Z2 board allows us to run a Linux-based operating system with Jupyter Notebook support, enabling FPGA programming, test execution, and metric collection. Additionally, it can run the software implementation of Kyber, allowing for direct comparisons between hardware and software performance. By the end of this guide, readers will be able to analyze resource utilization, power consumption, and junction temperature, as well as compare performance across software execution and the HLS-based accelerator.

2 Pre-requisites and Environment Setup

2.1 Required tools

To follow this tutorial, the following tools are required:

- **AMD Vitis 2024 or later**: it is an integrated development environment (IDE) is used to write the application code, apply high-level synthesis (HLS) pragmas, and translate C/C++ code into a hardware description.
- **AMD Vivado 2024 or later**: Vivado is used to import the IP cores generated by Vitis and to create the FPGA bitstream.
- **PYNQ-Z2 board** (or any board compatible with the PYNQ environment): it is the target hardware platform. The PYNQ-Z2 includes a Zynq SoC FPGA, which combines programmable logic with an ARM processor capable of running a Linux-based operating system. This system enables hardware control through Jupyter Notebook.
- **Ubuntu 24 or later**: The recommended host operating system for reliably running AMD/Xilinx development tools.
- **Peripherals**:
  - An **Ethernet cable** to connect the board to a local network and access the Jupyter Notebook server.

- A **MicroSD card** (minimum of 8 GB recommended) for flashing and booting the PYNQ Linux image.
- A **Micro-USB cable** for powering the board and enabling a serial terminal connection.

If necessary, the appendix shows how to install the required softwares.

2.2 Configuring the board

To set up the Jupyter environment, you need to download a disk imaging tool. We recommend using balenaEtcher, available at `https://etcher.balena.io/`. Next, download the PYNQ-Z2 SD card board image from `http://www.pynq.io/boards.html`.

Open the disk imaging tool, insert the SD card into your computer and unzip the image file that was downloaded earlier. In the software, select the inserted SD card and choose the path to the image. Click *Write* and wait until the process is finished. Finally, you can remove the SD card from your computer.

With the PYNQ-Z2 board, see the Figure 1 and make the following connections:

1. Connect the Ethernet cable from the board to a router or to an available Ethernet port on your computer;
2. Connect the micro-USB cable to a USB port on your computer;
3. Insert the microSD card with the installed image;
4. Set the power jumper so that the board is powered via the micro-USB cable;
5. Set the boot jumper so that the board boots from the microSD card;
6. Locate the board's power switch.

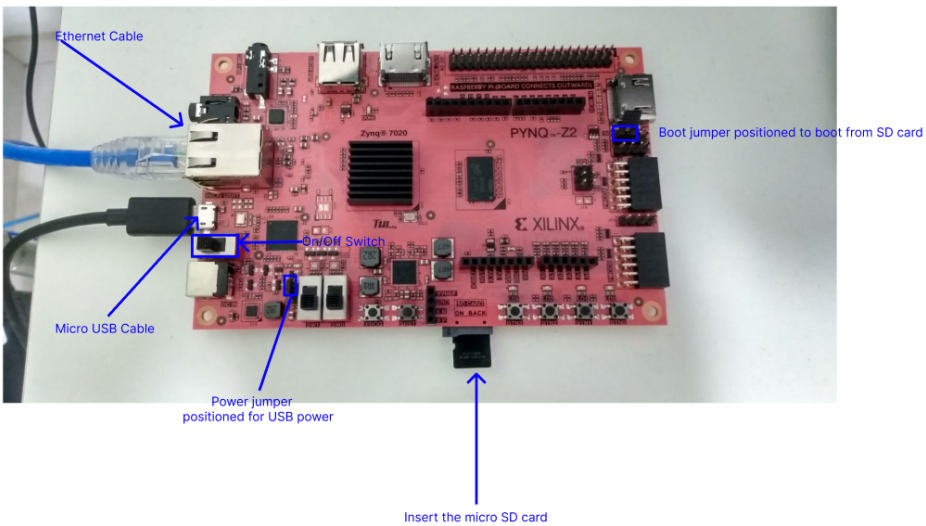

Figure 1: PYNQ-Z2 connections required for setting up the environment

Next, push the power switch to the *On* position. The CR8 LED (located in the lower left corner) should light up red. Wait a few seconds. The LD0, LD1, LD2, and LD3 LEDs should flash rapidly green, and the LD4 and LD5 LEDs will do the same in blue. Finally, the DONE LED located in the upper left corner will light up along with the LD0, LD1, LD2, and LD3 LEDs. Thus, the system will be booted and ready for use. The Figure 2 shows the board at the end of this process.

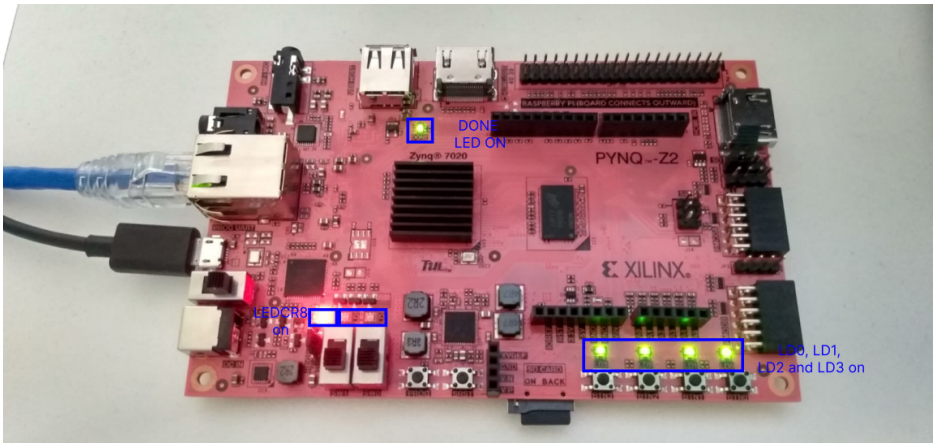

Figure 2: PYNQ-Z2 boot done

2.3 Accessing the Jupyter Notebook

To access the Jupyter environment, install a serial port terminal emulator, we will use the GTKTerm in this guide. Open a terminal and enter the following command:

```
sudo apt install gtkterm # To install the GTKTerm
```

Wait for the installation to finish and the software should display in the downloaded apps. Open it, click on *Configurations* → *Port* and a new tab will open. Choose the PYNQ-Z2 port, the port is `/dev/ttyUSB1` in our case, see the Figure 3.

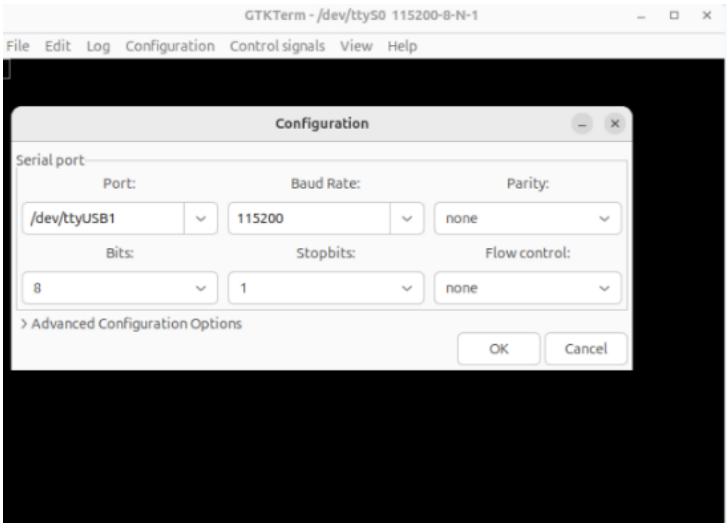

Figure 3: Configuring the PYNQ-Z2 serial port

Now, it is possible to open a terminal in the operational system running on PYNQ. Therefore, type the command bellow to find the IP address of the board:

```
ifconfig
```

Identify the address in `eth0` → `inet`. The Figure 4 shows how to identify the IP address and copy it.

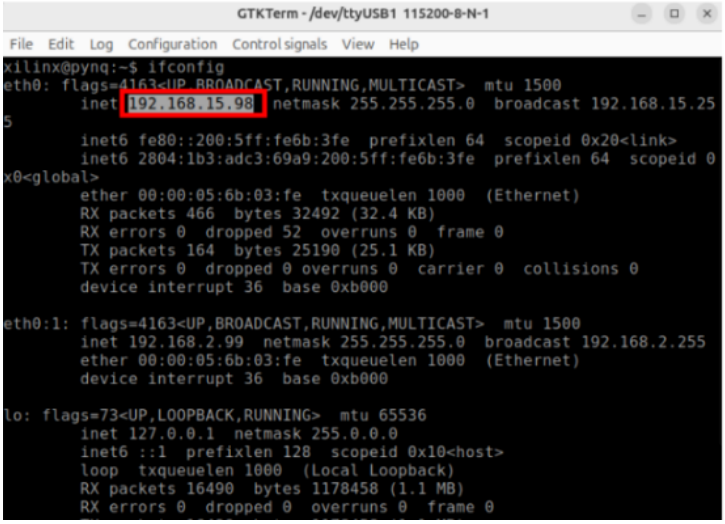

Figure 4: Identifying the IP address from PYNQ-Z2 board

Open a web browser (preferably Google Chrome or Firefox) and enter the address copied earlier in the search bar. The Jupyter page will open and it will ask the password. The default password is `xilinx`.

3 HLS Implementation Strategy

3.1 Selected Operations for Acceleration

We identified ten computationally intensive operations that are repeatedly invoked during the Kyber key encapsulation process and targeted them for acceleration. Their descriptions are presented below:

1. Vector Operations on Polynomials
   - `POLYVEC_NTT`: applies the NTT in all elements of a vector.
   - `POLYVEC_INVNTT_TOMONT`: executes the inverse NTT on a vector, returning the coefficients to the normal domain.
   - `POLYVEC_BASEMUL_ACC_MONTGOMERY`: performs coefficient by coefficient multiplication of two vectors of polynomials that are already in the NTT domain, accumulating the result into a single polynomial and applying Montgomery's reduction.
   - `POLYVEC_ADD`: performs element-by-element addition between two vectors of polynomials without modular reduction.
   - `POLYVEC_REDUCE`: applies Barrett's reduction to all coefficients of the polynomial vector.
2. Operations on Simple Polynomials
   - `POLY_TOMONT`: converts all the coefficients of a polynomial from the normal domain to the Montgomery domain.
   - `POLY_INVNTT_TOMONT`: applies the inverse NTT in a single polynomial to return to normal domain and applying the Montgomery multiplication factor.

- `POLY_ADD`: add the coefficients of two polynomials in parallel.
- `POLY_SUB`: subtract the coefficients of two polynomials in parallel.
- `POLY_REDUCE`: applies Barrett's reduction individually to the coefficients of a single polynomial.

3.2 Creating the top-level entity

Every HLS kernel must have a top-level entity that will later be synthesized into hardware. Since we have selected the operations to be accelerated, we can encapsulate them into a module responsible for controlling their execution. This module receives the necessary inputs along with a control signal that determines which operation the kernel should execute.

The top-level module, defined by the `polyvec_accel` function, acts as the communication interface between the Processing System (the ARM processor) and the Programmable Logic (PL) of the FPGA [8]. The communication interfaces were configured at two distinct levels:

1. **Control Interface** (AXI4-Lite): responsible for defining the IP control signals (start, done, idle, ready) and handling the transfer of scalar parameters (operation selector) [2]. These signals are implemented using the AXI4-Lite bus that allows low-latency transfer for simple transactions, ideal to register mapping. The `#pragma HLS INTERFACE s_axilite` directive is used for these signals.

2. **Data Interface** (AXI4-Master): responsible for communication with the external RAM, where the data resides [2]. It is performed via the AXI4-Master bus. This protocol is typically used for large volumes of data, such as the individual polynomials or the vectors of polynomials in our case. To handle the data load, burst transfer parameters were configured, allowing up to 16 pending transactions. Additionally, pointers were separated into independent channels (`bundle=gmem0, gmem1, gmem2`), enabling simultaneous read and write operations. The `#pragma HLS INTERFACE m_axi` directive is used for these signals.

Individual polynomials consist of `KYBER_N` (256) 16-bit coefficients, totaling 512 bytes. Polynomial vectors contain `KYBER_N` (256) $\times$ `KYBER_K` (3) 16-bit coefficients, requiring the transfer of 1536 bytes. To mitigate data transfer latency, we propose an architecture based on the *Load-Compute-Store* paradigm to isolate external memory accesses. The processing flow follows a three sequential steps.

In the load step, the input data allocated in exterminal memory is copied in its entirety to local memories in the FPGA. The Vitis HLS compiler automatically infers burst transfers from this function. In the case of a vector operation on the Kyber-768, this means pulling contiguous blocks of 1536 bytes directly into the Block RAMs (BRAMs) continuously and highly efficiently.

During the compute phase, the selected operation is executed on the local memories. Since the data resides in FPGA memory, it can be partitioned for parallel access, allowing the accelerator's datapath to be fed continuously without stalls caused by DDR memory latency.

Finally, after the calculations are completed, the result written to local memory is transferred back to the destination address in external memory. This step packages the results into a new write burst transaction.

The Figure 5 illustrates the proposed architecture for the accelerator.

3.3 Applying Optimizations to the Operations

This subsection explains the choice of optimizations used throughout the operations.

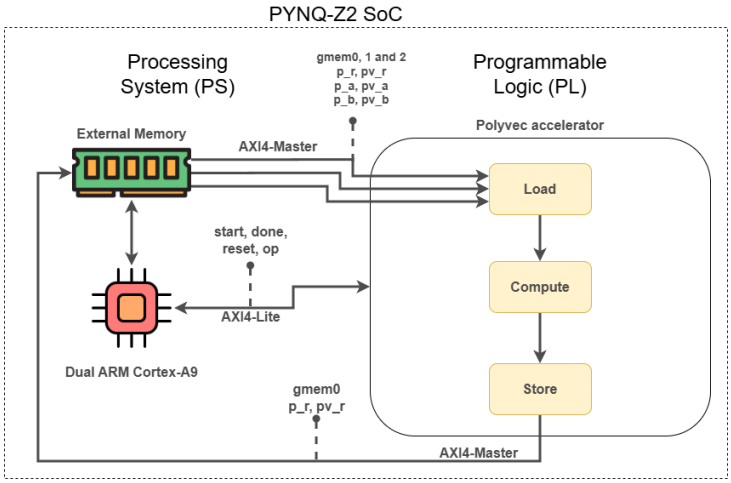

Figure 5: Architecture proposed for the accelerator

3.3.1 Array Partitioning

The Number-Theoretic Transform (NTT) and its inverse require simultaneous access to multiple coefficients during butterfly operations. In a standard implementation, vectors are mapped to block RAMs (BRAMs) with a limited number of read and write ports (typically two), creating an access bottleneck.

To solve this issue, we apply the `#pragma HLS ARRAY_PARTITION` directive to the input vectors of the NTT and inverse NTT functions, as well as to the higher-level functions `polyvec_ntt` and `polyvec_invntt_tomont`. The partitioning is performed cyclically with a factor of 16, meaning that vector elements are interleaved across 16 distinct BRAM banks. This approach increases the internal memory bandwidth, allowing the datapath to read and write multiple coefficients per clock cycle, thereby supporting the high degree of parallelism required for NTT iterations.

The Figure 6 illustrates how the cyclic array partitioning with a factor of 16 is applied to a vector.

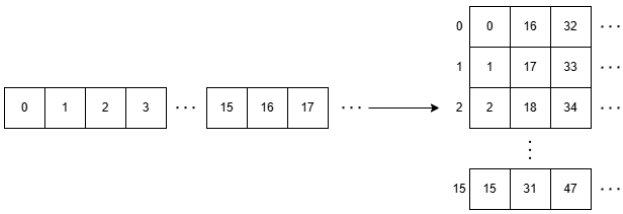

Figure 6: Cyclic Array Partitioning with a factor of 16

3.3.2 Pipeling

The pipelining technique was widely applied to the inner loops of the algorithms, including the NTT, inverse NTT, and vector addition and subtraction and other operations. This is implemented via the `#pragma HLS PIPELINE` directive. The *Initiation Interval* (II) is set to 1, which instructs the synthesis tool to generate a datapath in which a new loop iteration begins every clock cycle. This overlaps the execution of operations and prevents hardware from remaining idle while waiting for one operation to complete before the next begins.

The Figure 7 shows a comparison between a sequential loop execution (A) and a pipelined loop (B).

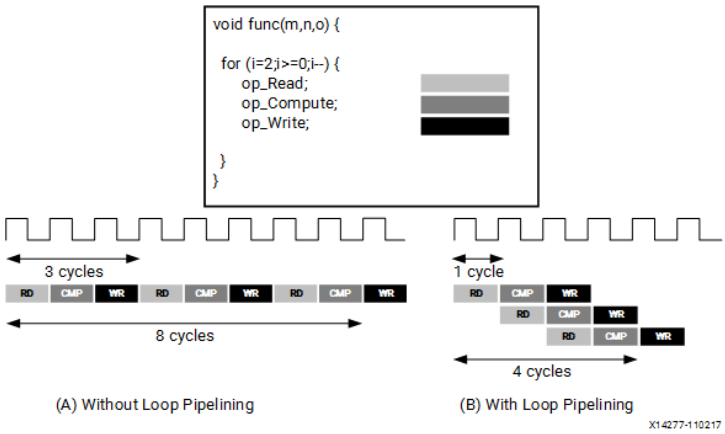

Figure 7: Comparison between sequential and pipelined loop execution (**Source:** Vitis High-Level Synthesis User Guide (2025))

3.3.3 Loop Unrolling

Loop unrolling was applied to `verify` and `cmov` functions. This technique replicates the physical logic of the loop body, creating multiple parallel processing paths. We can set the unroll factor that specifies to the synthesis tool how many data elements should be processed simultaneously in a single clock cycle. By unrolling the loop, the tool generates multiple instances of the combinational logic, reducing the number of sequential iterations and increasing throughput at the cost of additional hardware resources. This optimization is implemented by the directive `#pragma HLS UNROLL`.

3.3.4 Inlining Functions

Modularizing C code makes it easier to read, but in hardware it introduces overhead due to the creation of additional Finite State Machines (FSMs) to manage subroutine calls.

To deal with this, we can mark frequently called functions, such as modular arithmetic and basic polynomial operations, with the `#pragma HLS INLINE`. This directive instructs the compiler to "flatten" the entire call hierarchy, inserting the logic circuit of the invoked function directly into the caller block. This approach not only saves control cycles but also allows resource sharing (such as DSP blocks) between operations.

4 RTL and Bitstream Generation

4.1 High-Level Synthesis and IP Export (Vitis HLS)

Open the Vitis software and click on *Create New Component* in the HLS Development tab. Select *Create Empty HLS Component*, then enter the desired accelerator name and choose the directory where the project will be created. Click *Next* twice.

On the Hardware tab, add the project source files and the testbench file (`polyvec_accel_tb.c`). Also select the top function, which is `polyvec_accel`. Click *Next*. Then, choose the target FPGA, in our case the PYNQ-Z2, by selecting the part `xc7z020clg400-1`. Finally, complete the project creation.

On the left side of the screen, you can see the components we add before and the *Flow* options that we will use to synthetize our project, see figure 8.

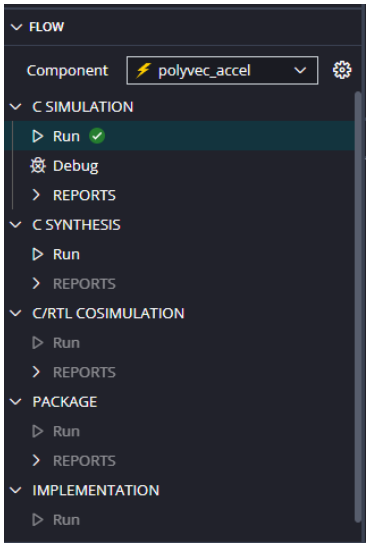

Figure 8: Vitis design flow tab

The function of each of the options is as follows:

1. **C SIMULATION:** Verifies the functional correctness of the original C/C++ algorithm using a testbench prior to any hardware synthesis.
2. **C SYNTHESIS:** Transforms the C/C++ source code into a Register Transfer Level (RTL) hardware description, such as Verilog or VHDL.
3. **C/RTL COSIMULATION:** Validates the generated RTL design by reusing the original C testbench to ensure functional equivalence between the software and hardware models.
4. **PACKAGE:** Packages the synthesized RTL design into an Intellectual Property (IP) block, making it ready to be exported to the Vivado IP Catalog.
5. **IMPLEMENTATION:** Performs logic synthesis and place-and-route on the RTL design to provide accurate timing and resource utilization estimates.

Now, you can run each option one by one to complete the high-level synthesis and perform functional verification of the code. Check whether any errors occur during the process. In the end, the RTL should be successfully generated, allowing you to import the design to Vivado.

It is important to note that the implementation step takes a long time and is optional, since its results can be obtained more accurately later in Vivado.

4.2 System Integration and Block Design (Vivado)

Open the Vivado software and click on *Create Project*, then enter the desired project name and choose the directory where the project will be created. Click *Next* and select the project type as *RTL Project*. Then, choose the target board, in this case, the PYNQ-Z2 or select the part `xc7z020clg400-1`. Finally, complete the project creation.

The figure 9 shows the Vivado flow that you can see on the left side of the screen.

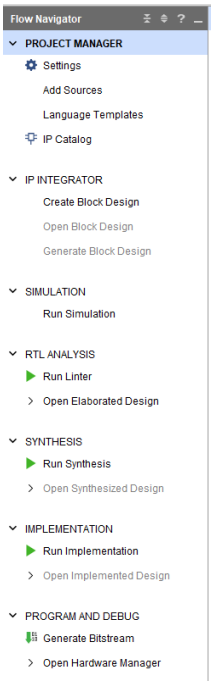

Figure 9: Vivado design flow tab

We will use the accelerator IP created in Vitis through the *IP Catalog* option. This will open a new tab. Right-click on *Vivado Repository* and select *Add Repository*. Then, choose the Vitis component directory. Vivado will indicate that the IP has been successfully added to the project.

Click on Block Design in the Vivado Flow Navigator to open the canvas. Once it appears, right-click anywhere on the canvas and select *Add IP*. Add the following components: the *polyvec accelerator* and the *ZYNQ7 Processing System*.

Next, run both the *Connection Automation* and the *Block Automation*. After that, double-click on the *processing system*. In the *PS-PL Configuration*, enable the *S AXI HP0* interface to allow data transfer through the AXI high-performance interface.

Run the *Connection Automation* again to connect the three bundles previously defined in the HLS design. The final Block Design should resemble the one shown in the figure 10.

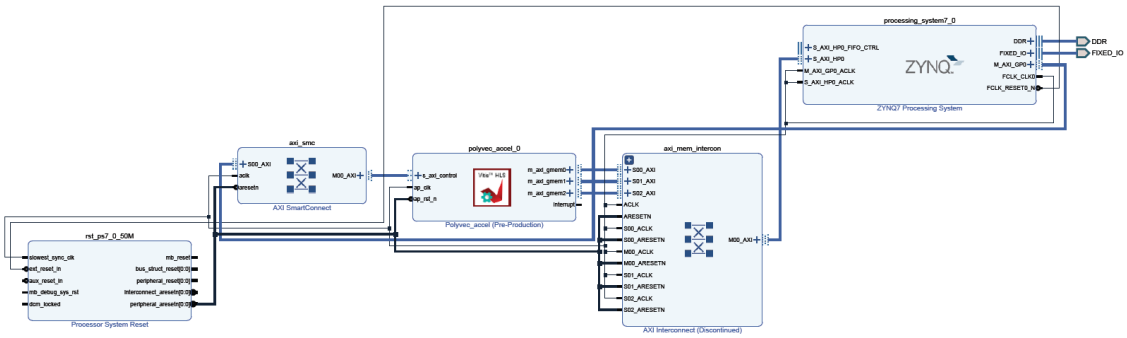

Figure 10: Vivado block design of the Polyvec accelerator

Go to the *Sources* tab. Under *Design Sources*, right-click the block file and select

Create HDL Wrapper to encapsulate the block design into a single top-level module.

Finally, you can either run *RTL Analysis*, *Synthesis*, *Implementation*, and *Generate Bitstream* individually, or simply run *Generate Bitstream*, which will automatically execute all the preceding steps. Wait for the process to finish.

To run the accelerator, we need to obtain the bitstream file and the hardware handoff file (.hwh). The hardware handoff file is typically located at a path like `<project_directory>/<project_name>.gen/sources_1/bd/<block_design_name>/hw_handoff`, while the bitstream can be found at `<project_directory>/<project_name>.runs/impl_1/<top_module_name>.bit`.

Additionally, rename either the bitstream or the hardware handoff file so that both share the same name.

5 Executing the Design and Obtaining Metrics

After the bitstream is generated, Vivado shows a report with the average temperature, resources utilization, power and other informations. Tables 1 and 2 summarize the power dissipation per resource, the thermal characteristics of the design, and the resource utilization of the accelerator.

Table 1: Power Analysis and Thermal Estimation from Implemented Netlist

Resource	Power	Percentage
Total On-Chip Power	**1.599W**	**100%**
Dynamic Power	1.458W	91%
Device Static Power	0.141W	9%
Dynamic Power Breakdown		
PS7	1.260W	86%
Signals	0.053W	4%
Clocks	0.052W	4%
Logic	0.034W	2%
DSP	0.032W	2%
BRAM	0.028W	2%
Thermal Information		
Junction Temperature	43.4 °C	–

Table 2: Hardware Resource Utilization for the Accelerator ($N = 256, K = 3, Q = 3329$)

Resource	Utilization	Available	Utilization %
LUT	13984	53200	26.29%
LUTRAM	1509	17400	8.67%
FF	14274	106400	13.42%
BRAM	15.5	140	11.07%
DSP	91	220	41.36%
BUFG	1	32	3.13%

To run the accelerator on the FPGA, open the Jupyter environment as described in Section 2.3. Create a new folder and upload the bitstream, the hardware handoff and the notebook code. The notebook is structured into six main steps:

1. **Initialization and Hardware Configuration:** sets up the Python environment and programs the FPGA. This step imports the necessary libraries and loads the bitstream file onto the FPGA. Make sure the *Overlay* command points to the correct path of the bitstream.

2. **Software Reference Preparation:** to verify the hardware results, a reliable software reference is executed for comparison. The notebook runs a `gcc` command in the terminal to compile the original Kyber implementation in C, producing a shared library. This library is then loaded by the notebook, which maps the C functions so they can be called directly from Python.

3. **Parameters and Auxiliary Functions:** this section establishes the rules and communication protocols for the system. Defines constant like `KYBER_N` and `KYBER_Q` and maps each mathematical operation to a numeric code, which instructs the hardware which data path to activate. Also defines control functions to manage communication between the processor and the FPGA via AXI-Lite registers.

4. **Shared Memory Management:** instead of using standard numpy arrays, the notebook uses PYNQ's allocate function. This creates blocks of contiguous physical memory in RAM that can be accessed directly by both the ARM processor and the FPGA. The physical memory addresses for the input arrays and the output array are sent to the hardware accelerator's registers using the auxiliary functions created earlier.

5. **Validation Tests:** the script runs 10000 iterations for each supported mathematical operation. In each iteration, random inputs are generated, the operation is executed in software (C), and then on the hardware (FPGA). The outputs are compared using modular arithmetic (diff % KYBER_Q). Any discrepancies are logged, and the final report confirms that the hardware design faithfully reproduces the original algorithm.

6. **Benchmarking:** in this final step, the notebook measures the execution time for both the software implementation and the hardware execution across all 10000 iterations. The result is a comparative table.

Run the cells sequentially from top to bottom using Shift + Enter, or select Kernel → Restart & Run All.

The C compilation step runs silently if successful. The *Validation Tests* cell reports whether all 10,000 iterations pass without errors. Finally, the *Performance Evaluation* cell displays the benchmarking table, showing the FPGA speedup over the ARM processor.

The execution times and the resulting speedup factors for the implemented operations are detailed in Table 3

Table 3: Performance Comparison and Speedup: ARM Cortex-A9 vs. HLS Accelerator

Operation	**SW Time (μs)**	**HW Time (μs)**	**Speedup**
POLYVEC_NTT	1969.21	241.56	8.15×
POLYVEC_INVNTT_TOMONT	2419.85	245.15	9.87×
POLYVEC_BASEMUL_ACC_MONT	777.30	122.59	6.34×
POLYVEC_ADD	174.93	122.21	1.43×
POLYVEC_REDUCE	253.10	120.55	2.10×
POLY_TOMONT	112.06	99.81	1.12×
POLY_INVNTT_TOMONT	838.24	140.73	5.96×
POLY_ADD	89.57	95.79	0.94×
POLY_SUB	91.18	97.02	0.94×
POLY_REDUCE	113.97	99.92	1.14×

6 Discussion of Results

Based on table 1, the proposed design exhibits high energy efficiency. The total on-chip power consumption is estimated at 1.599W, of which 91% (1.458W) corresponds to dynamic power. The Processing System accounts for the majority of the dynamic power dissipation, contributing 1.260W (86%), whereas the Programmable Logic (PL) represents a comparatively small fraction. The junction temperature remains stable at $43.4°C$, indicating that the system operates within safe thermal limits without requiring additional cooling mechanisms.

The resource utilization results presented in table 2 indicate a well-balanced design with sufficient margin for future extensions. DSP blocks exhibit the highest utilization at 41.36%, which is consistent with the computational demands of polynomial multiplication and modular arithmetic. In contrast, LUTs (26.29%), Flip-Flops (13.42%), and BRAM (11.07%) show moderate utilization levels, demonstrating the effectiveness of the adopted array partitioning and pipelining strategies.

The performance evaluation in table 3 highlights the trade-offs inherent to hardware acceleration in heterogeneous SoC architectures. Significant speedups are achieved for computationally intensive operations, reaching 9.87x for `POLYVEC_INVNTT_TOMONT` and 8.15x for `POLYVEC_NTT`. Similarly, operations involving Montgomery accumulation achieve a speedup of up to 6.34x.

On the other hand, lower-complexity operations such as `POLY_ADD` and `POLY_SUB` exhibit a slight performance degradation (0.94x). This behavior suggests that, for such operations, the overhead associated with data transfer between external memory and on-chip BRAM outweighs the benefits of parallel execution in the PL. These results indicate that hardware acceleration is particularly advantageous for complex vector operations in the NTT domain, while simpler arithmetic operations are more efficiently executed on the ARM processor.

Appendix – Installing Vitis and Vivado 2025.2

This appendix shows how to install *Vitis* and *Vivado* 2025.2.

Installing Vitis and Vivado

Go to the link https://www.xilinx.com/support/download.html and download the unified installer *AMD Unified Installer for FPGAs & Adaptive SoCs 2025.2*. You must register in AMD website and save your login and password, because it is going to be necessary later.

Open a new terminal and go to the directory that the installer was downloaded. Type the following commands:

```
chmod +x FPGAs_AdaptiveSoCs_Unified_SDI_2025.2_1114_2157_Lin64.bin
./FPGAs_AdaptiveSoCs_Unified_SDI_2025.2_1114_2157_Lin64.bin
```

After running the commands, the installer should open. Click *Next* once, then enter the same email address and password you used to create your AMD account. Click *Next* again, select the *Vivado* option and then choose *Vivado ML Standard*. The Figure 11 shows the required installation options.

Agree with the terms and choose the installation directory (keep the default `/tools/Xilinx`). Click *Next* and wait for the installation to finish. In this step, it is common to occur the error: *"Cannot write to /tools/Xilinx"*, if that happens, open a new terminal and enter the lines:

```
cd / # To be directed to the / directory
sudo mkdir tools # To create the tools directory
```

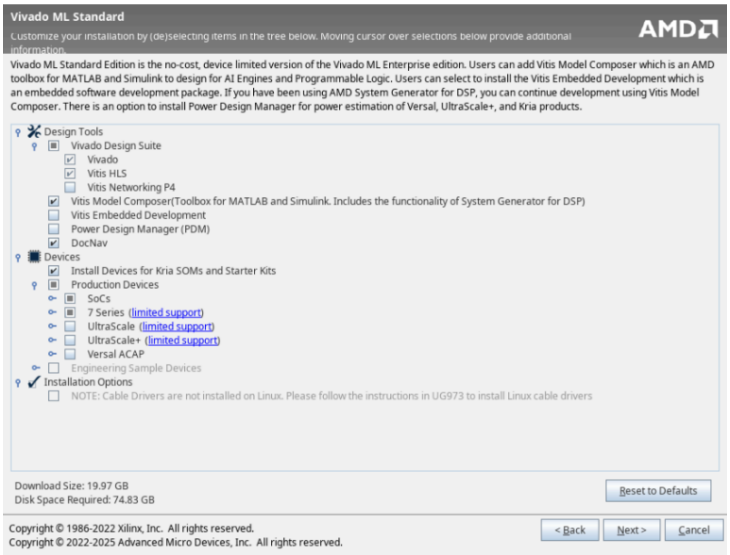

Figure 11: Devices options

```
sudo mkdir tools/Xilinx # To create the Xilinx directory
sudo chmod ugo\+rwx tools/Xilinx # To give the installer read and write permission
```

Go back to the installer, click on *Back* to return one step, then *Next* and select the installer directory. The installer should no longer indicate the error and it will be possible to proceed with the installation.

Importing boards

When installing Vivado, several boards are not shown during project creation, in our case, the PYNQ-Z2. Therefore, an additional step is required to install the PYNQ-Z2 board or other boards. Clone the Github repository from the following link as a *zip* file: https://github.com/Xilinx/XilinxBoardStore/tree/2022.2.

Extract the *zip* file to a folder. Navigate to the directory `XilinxBoardStore-2022.2/boards/TUL` and copy the PYNQ-Z2 folder. Go to the directory `/tools/Xilinx/Vivado/2024.2/data/xhub/boards/XilinxBoardStore/boards`. Create a directory called `TUL` and paste the PYNQ-Z2 folder copied earlier.

Reopen Vivado, create a new project, and confirm that the PYNQ-Z2 board is available during project creation. Figure 12 shows the Vivado interface during the project setup, with the PYNQ-Z2 board selected.

You can add one board at a time or add all available boards from the Github repository. If you wish, copy all boards from `XilinxBoardStore-2022.2/boards/` and paste them into `/tools/Xilinx/Vivado/2024.2/data/xhub/boards/XilinxBoardStore/boards`.

Installing USB Drivers

After installing Vivado, the drivers for recognizing devices on the USB port must be installed. To do this, enter the following commands in a terminal:

```
cd /tools/Xilinx/Vivado/2024.2/data/xicom/cable_drivers/lin64/install_script/
    install_drivers # to navigate to the folder containing the driver installer
sudo ./install_drivers # to install the drivers with the necessary permissions
```

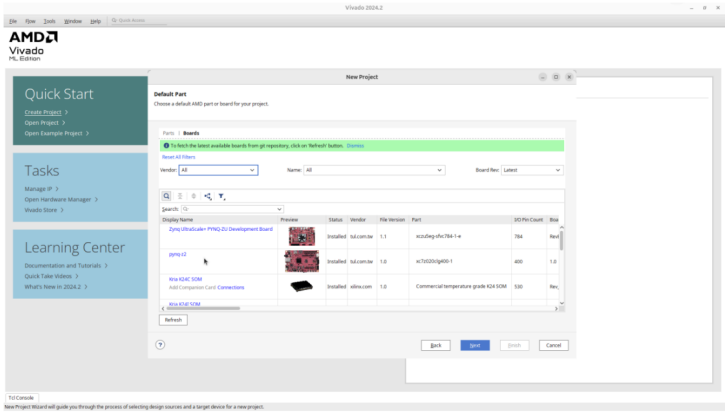

Figure 12: PYNQ-Z2 board during the project creation

Close Vivado and disconnect the USB ports (if open and connected, respectively) and reopen and reconnect.

References

[1] Advanced Micro Devices, Inc. *Vitis High-Level Synthesis User Guide (UG1399)*. AMD, San Jose, CA, jan 2026. v2025.2.

[2] ARM Limited. *SoC Designer AXI4 Protocol Bundle User Guide*, 2016.

[3] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '96, page 212–219, New York, NY, USA, 1996. Association for Computing Machinery.

[4] R. Kastner, J. Matai, and S. Neuendorffer. Parallel Programming for FPGAs. *ArXiv e-prints*, May 2018.

[5] Dustin Moody, Ray Perlner, Andrew Regenscheid, Angela Robinson, and David Cooper. Transition to post-quantum cryptography standards. Technical Report NIST IR 8547 ipd, National Institute of Standards and Technology, Gaithersburg, MD, nov 2024.

[6] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '05, page 84–93, New York, NY, USA, 2005. Association for Computing Machinery.

[7] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997.

[8] TUL Corporation. *PYNQ-Z2 Reference Manual*. TUL Corporation, may 2018. v1.0.